

Capítulo 3 - Making Decisions

Pregunta 1

- **Respuestas correctas:** A, B, C, E, F, G
- **Explicación:** Un `switch` solo acepta números enteros normales o pequeños (`int`, `byte`, `short`, `char`), sus clases envolventes (`Integer`, `Byte`, etc.), textos (`String`), opciones de un `enum` y variables `var` que representen estos tipos. Tipos como `long`, `float` o `double` están prohibidos.

Pregunta 2

- **Respuesta correcta:** B
- **Explicación:** La primera condición se cumple porque 4 es mayor o igual a 4. Pero adentro, la segunda condición falla porque la humedad vale 8 y no es menor a 6. Por eso salta directamente al `else` interno e imprime "**Just Right**".

Pregunta 3

- **Respuestas correctas:** A, D, F, H
- **Explicación:** El bucle `for-each` solo sirve para recorrer arreglos o colecciones de elementos ordenados (como `List` o `Set`). Los mapas (`Map`) y los objetos genéricos no se pueden recorrer directamente con este bucle.

Pregunta 4

- **Respuesta correcta:** F
- **Explicación:** Cuando usas un `switch` para guardar un resultado en una variable, estás obligado a cubrir todas las posibilidades. Como faltó la opción por defecto (`default`), el programa no compila.

Pregunta 5

- **Respuesta correcta:** E
- **Explicación:** En el segundo bucle pones un `continue;` directo sin ninguna condición. Esto hace que la línea de abajo sea imposible de alcanzar (código inalcanzable), algo que Java no permite.

Pregunta 6

- **Respuestas correctas:** C, D, E
- **Explicación:** **C** es cierta porque en un `for` la condición se revisa antes de empezar. **D** es correcta porque un `switch` que devuelve un valor requiere obligatoriamente un

`default`. E es verdadera porque el bucle `do/while` siempre corre al menos una vez antes de evaluar la condición.

Pregunta 7

- **Respuestas correctas:** B, D
- **Explicación:** Para recorrer un arreglo de inicio a fin avanzas del índice 0 a la última posición (B). Para ir en reversa, empiezas en la última posición y vas restando hasta llegar a cero (D).

Pregunta 8

- **Respuesta correcta:** G
- **Explicación:** Tiene dos errores: no puedes usar la variable `bat` después de un operador `||` porque no hay garantía de que exista si la primera parte fue falsa, y la palabra `default` no existe dentro de una estructura `if/else`.

Pregunta 9

- **Respuestas correctas:** B, C, E
- **Explicación:** Las tres opciones interrumpe o saltan el bucle interno cuando se cumple la condición de número par, haciendo que el contador solo incremente en las vueltas impares hasta sumar un total de 2.

Pregunta 10

- **Respuesta correcta:** E
- **Explicación:** Contiene 4 líneas con error: no puedes usar `continue` suelto dentro de un `switch`, las opciones del `case` deben ser valores fijos en lugar de variables normales, y los tipos de datos deben coincidir con lo que evalúa el `switch`.

Pregunta 11

- **Respuesta correcta:** A
- **Explicación:** El código funciona bien. Al pasarle la opción `MAMMAL`, el `switch` entra a esa rama, asigna el número 3 a la variable e imprime 3.

Pregunta 12

- **Respuesta correcta:** C
- **Explicación:** En la primera vuelta suma $7 + 4 = 11$. En la segunda vuelta suma $6 + 6 = 12$ (acumulado 23). En la tercera vuelta la condición de que el primer número sea mayor al segundo falla, sale del bucle e imprime 23.

Pregunta 13

- **Respuesta correcta:** G

- **Explicación:** No compila porque la condición al final del bucle `do/while` debe ir obligatoriamente entre paréntesis: `while (condicion);`.

Pregunta 14

- **Respuestas correctas:** B, D, F
- **Explicación:** El bucle `for-each` detecta automáticamente el tipo de elemento individual dentro del grupo que recorre: de un arreglo `int[]` obtiene `int`, de `Character[]` obtiene `Character`, y de una lista `List<Integer>` obtiene `Integer`.

Pregunta 15

- **Respuesta correcta:** F
- **Explicación:** No compila porque la sintaxis `case 'B': 'C':` es incorrecta. En Java los casos múltiples se separan por comas (`case 'B', 'C':`) o se repite la palabra `case`.

Pregunta 16

- **Respuestas correctas:** A, B, D
- **Explicación:** Las tres opciones correctas arrancan desde la última posición del arreglo y van restando el índice hasta llegar a 0 sin salirse de los límites.

Pregunta 17

- **Respuestas correctas:** B, E
- **Explicación:** El primer bucle avanza sumando hasta llegar a 10. El segundo bucle corre una vez y deja la variable en 3. Los dos únicos números distintos que se imprimen en pantalla son **10** y **3**.

Pregunta 18

- **Respuestas correctas:** C, E
- **Explicación:** La comprobación de tipos moderna usa la palabra clave `instanceof` (C) y activa la variable automáticamente solo en las líneas donde es seguro que pertenece a ese tipo (E).

Pregunta 19

- **Respuesta correcta:** E
- **Explicación:** La variable `snake` se creó adentro del bloque del `do`. Cuando la condición del `while` intenta leerla afuera, no la encuentra y da error de compilación por estar fuera de alcance.

Pregunta 20

- **Respuestas correctas:** A, E
- **Explicación:** El bucle interno es infinito. Para evitar que el programa se quede congelado, debes usar `break` L2 o `continue` L2 para saltar o romper el bucle etiquetado que está más afuera.

Pregunta 21

- **Respuesta correcta:** E
- **Explicación:** Tiene 4 errores: el `switch` no acepta variables `Long`, a la expresión le falta la palabra clave `yield`, sobra un punto y coma y hay un caso duplicado (`case 30`).

Pregunta 22

- **Respuesta correcta:** E
- **Explicación:** Al valer 3, entra al `case 3` e imprime 5. Luego entra al bucle `while` que resta antes de evaluar e imprime 2 y luego 1. La salida completa es **5 2 1**.

Pregunta 23

- **Respuesta correcta:** F
- **Explicación:** Hay un `else if` suelto. La línea anterior ya había cerrado la estructura con un `else` definitivo, por lo que poner otro `else if` inmediatamente después provoca un error de sintaxis.

Pregunta 24

- **Respuesta correcta:** G
- **Explicación:** Ninguna opción es válida porque el código usa la palabra `in`, la cual no existe en Java para recorrer elementos (en Java se usan dos puntos `:`).

Pregunta 25

- **Respuesta correcta:** D
- **Explicación:** Como busca "violin" en minúsculas y no coincide con "VIOLIN", cae al `default`, le suma 1 a la variable y luego sigue bajando por las opciones siguientes hasta encontrar un `break`, acumulando un total de **2**.

Pregunta 26

- **Respuesta correcta:** F
- **Explicación:** La variable que controla la condición del bucle de adentro se modifica afuera, por lo que la condición interna nunca cambia a falso y se queda en un bucle infinito al ejecutarse.

Pregunta 27

- **Respuesta correcta:** F
- **Explicación:** Cuando un `switch` devuelve un valor, debe garantizar que entregará algo en todos los caminos posibles. Aquí hay una condición `if` dentro de un caso que si da falso no devuelve nada, rompiendo la regla.

Pregunta 28

- **Respuesta correcta:** F
- **Explicación:** Falla al compilar porque intenta crear una variable nueva con el mismo nombre de otra variable que ya estaba activa en ese mismo bloque de código.

Pregunta 29

- **Respuesta correcta:** C
- **Explicación:** El bucle `do/while` primero suma 1 y luego imprime. Arranca en -2, así que imprime desde -1 subiendo de uno en uno hasta llegar a 6, donde detecta que superó el límite de 5 y se detiene.

Capítulo 4 - Core APIs

Pregunta 1

- **Respuesta correcta:** F
- **Explicación:** No compila en la línea 5. Sumas dos números enteros y el resultado es un número, pero intentas guardarlo directamente en una variable de texto (`String`) sin convertirlo.

Pregunta 2

- **Respuestas correctas:** C, E, F
- **Explicación:** Las tres declaraciones están mal escritas: la C escribe `new beans` en lugar de usar un tipo válido como `String`, la E olvida poner el tamaño dentro de los corchetes al crear el arreglo, y la F deja vacíos ambos corchetes.

Pregunta 3

- **Respuestas correctas:** A, C, D
- **Explicación:** Acepta fechas que sí existan en el calendario: el 13 de marzo de 2022 (A), el 6 de noviembre de 2022 (C) y el 7 de noviembre de 2022 (D). La B falla porque no existe el día 40, la E porque 2023 no fue bisiesto (febrero no tuvo 29 días), y la F porque `MonthEnum` es un tipo inventado.

Pregunta 4

- **Respuestas correctas:** A, C, D

- **Explicación:** Imprime "one", "three" y "four". Comprobar el texto directamente con `.equals()` da verdadero (A); usar `.intern()` busca el texto en la memoria compartida y da verdadero (C); y comparar el literal directo con la variable compartida también da verdadero (D).

Pregunta 5

- **Respuesta correcta:** B
- **Explicación:** Las operaciones en `StringBuilder` modifican el mismo texto de forma consecutiva: empieza con "aaa", inserta "bb" en la segunda posición quedando "abbbaa", e inserta "ccc" más adelante resultando en **abbaccca**.

Pregunta 6

- **Respuesta correcta:** C
- **Explicación:** las líneas 24 y 25 realizan asignaciones de tipos incompatibles sin un casteo explícito: en la línea 24, el método `Math.round(1.0)` recibe un valor `double` y devuelve un tipo `long`, el cual no se puede guardar directamente en una variable `int`; por su parte, en la línea 25, `Math.random()` devuelve un valor `double`, que tampoco se puede asignar a una variable `float` sin convertirlo previamente. Al faltar un casteo explícito en ambos casos, el código genera exactamente dos errores de compilación.

Pregunta 7

- **Respuesta correcta:** C
- **Explicación:** Para comparar fechas con distintas zonas horarias, lo ideal es convertirlas primero a la hora universal (GMT). La primera hora (05:00 GMT-04:00) equivale a las **09:00 GMT**, mientras que la segunda (09:00 GMT-06:00) equivale a las **15:00 GMT**. Al comparar ambas en el mismo huso horario, se observa que la primera ocurre antes (A) y que existe exactamente una diferencia de 6 horas entre las dos (E).

Pregunta 8

- **Respuestas correctas:** A, B, F
- **Explicación:** Buscar el carácter del índice 4 en "12345" da '5' (A). En B, al reemplazar parte del texto queda "1265", donde en la posición 3 está la '5'. En F, al reemplazar queda "145", donde en la posición 2 está la '5'.

Pregunta 9

- **Respuestas correctas:** A, C, F
- **Explicación:** Los arreglos siempre cuentan desde la posición 0 (A) y su tamaño es fijo (C). Como Java no compara el contenido interno de dos arreglos cuando usas

`.equals()`, simplemente revisa si son el mismo objeto en memoria, dando falso al ser dos arreglos distintos (F).

Pregunta 10

- **Respuesta correcta:** A
- **Explicación:** Cero líneas tienen error. Cada función matemática devuelve el tipo de dato correcto y todos esos valores entran sin problema dentro del arreglo de números decimales.

Pregunta 11

- **Respuesta correcta:** E
- **Explicación:** No compila. La clase `LocalDate` guarda únicamente fechas (año, mes, día), por lo que no existe ningún método `.plusHours()` para sumarle horas.

Pregunta 12

- **Respuestas correctas:** A, D, E
- **Explicación:** `.indent()` y `.stripLeading()` se anulan entre sí. La primera llamada a `.substring()` saca "12" (A), la segunda corta en el mismo sitio y devuelve texto vacío creando una línea en blanco (E), y la tercera toma el resto resultando en "78" (D).

Pregunta 13

- **Respuesta correcta:** B
- **Explicación:** las cadenas de texto de tipo `String` son inmutables, por lo que invocar `roar1.concat("!!!")` genera un nuevo texto pero deja la variable `roar1` intacta con su valor original ("roar"). Por el contrario, los objetos de tipo `StringBuilder` sí son mutables, lo que significa que el método `roar2.append("!!!")` modifica directamente el contenido en memoria de `roar2` cambiándolo a "roar!!!". Al imprimir ambas variables concatenadas, el resultado final es "roar roar!!!".

Pregunta 14

- **Respuestas correctas:** A, F
- **Explicación:** `Instant.now()` crea un instante de tiempo directamente (A). Un objeto de fecha con zona horaria completa (`ZonedDateTime`) se puede convertir a un instante con `.toInstant()` (F). Una fecha u hora por sí sola no tiene zona horaria para saber qué instante preciso representa.

Pregunta 15

- **Respuestas correctas:** C, E

- **Explicación:** Al ordenar el arreglo, los números van primero, luego las mayúsculas y al final las minúsculas: `["123", "PIG", "pig"]` (C). Al buscar `"Pippa"` dentro del arreglo ordenado, como no existe, devuelve un número negativo que indica la posición donde debería encajar (`-3`) (E).

Pregunta 16

- **Respuestas correctas:** A, B, G
- **Explicación:** La cadena original mide 11 caracteres (B). Al aplicar sangría con `.indent()`, se normaliza el texto aumentando a 16 caracteres (G). Finalmente, al procesar los caracteres especiales con `.translateEscapes()`, la longitud resultante es 10 (A).

Pregunta 17

- **Respuestas correctas:** A, G
- **Explicación:** `.substring(1, 2)` toma un solo carácter y devuelve una cadena válida (A). Si le pones un índice inicial mayor al índice final (como de 6 a 5), el programa falla y lanza una excepción de límites (G).

Pregunta 18

- **Respuestas correctas:** C, F
- **Explicación:** Los textos normales son inmutables, así que las modificaciones sueltas a `s1` se ignoran. Solo la concatenación `+= "two"` cambia la variable dejándola con 7 caracteres (C). Por otro lado, la variable `s2` se construye con el texto `"2cfalse"`, por lo que `.equals()` confirma que el contenido es idéntico e imprime `"equals"` (F).

Pregunta 19

- **Respuestas correctas:** A, B, D
- **Explicación:** `Arrays.compare()` devuelve un número positivo si el primer arreglo es alfabéticamente mayor en el primer elemento distinto (A). `Arrays.mismatch()` devuelve el índice positivo donde encuentra la primera diferencia entre los dos arreglos (B y D).

Pregunta 20

- **Respuestas correctas:** A, D
- **Explicación:** Cuando llega el fin de semana de adelanto de hora en primavera, el reloj salta de la 1:59 AM directamente a las 3:00 AM (desaparece la hora 2:00 AM). Por eso, sumarle una hora a la 1:30 AM da las 3:30 AM (A) y la hora del objeto pasa a ser las 3 (D).

Pregunta 21

- **Respuestas correctas:** A, C
- **Explicación:** El método `.reverse()` invierte directamente el texto del `StringBuilder` dejando "avaJ" (A). La opción C también logra dejar exactamente "avaJ" al agregar texto y luego recortar las partes sobrantes por los extremos.

Pregunta 22

- **Respuesta correcta:** A
- **Explicación:** Las fechas en Java son inmutables. El código calcula `.plusDays(2)` y `.plusYears(3)`, pero no guarda el resultado en ninguna variable, por lo que la fecha original nunca cambia y vuelve a imprimir **2022 APRIL 30**.